Study Report on Regret Matching and Counterfactual Regret Minimization

March 7, 2026

Abstract

This report summarizes my study of extensive-form games (EFGs), tree-form sequential decision processes (TFSDPs), regret matching (RM), and counterfactual regret minimization (CFR). I also report two small experiments: RM/RM+ on Rock-Paper-Scissors and CFR/CFR+ on Kuhn Poker [1]. The main goal was not only to implement the algorithms, but also to clarify several notation and modeling details that were initially confusing when reading different sources.

1 Introduction

1.1 EFGs and TFSDPs

An **extensive-form game (EFG)** describes a sequential game as a tree. Histories correspond to nodes in the tree, terminal histories carry utilities, and each non-terminal history belongs either to chance or to one of the players. Information sets encode imperfect information: if two histories belong to the same information set of player i, then player i cannot distinguish them when choosing an action. In two-player zero-sum games, the objective is to find a strategy profile close to a Nash equilibrium.

A **tree-form sequential decision process (TFSDP)** is a player-centric representation of sequential decision making. Instead of storing the full game tree at once, it describes a single player's local structure using observation points, decision points, signals, actions, and transition maps. In the implementation used for this project, a TFSDP is represented by

$$(J, A, K, S, \rho, \Sigma, p),$$

where J is the set of decision points, A_j is the action set at $j \in J$, K is the set of observation points, S_k is the signal set at $k \in K$, ρ is the transition map, Σ is the set of sequences, and p_j denotes the parent sequence of decision point j. Strictly speaking, Σ and p_j can be constructed from the other components, so they are included here mainly for convenience. In practice it is useful to store them explicitly because sequence-form strategy construction and CFR updates refer to them repeatedly.

One modeling detail that was initially confusing is that the same strategic game can have multiple equivalent EFG or TFSDP representations. For example, in player 1's TFSDP for Kuhn Poker, consider the branch in which player 1 is dealt Q, checks, and then the opponent also checks so that the game ends at showdown. One representation can end immediately at a terminal node with a variable utility equal to the expected payoff against the opponent's hidden card, which depends on the conditional probability that the opponent's hidden card is K or J given the public history. Another representation can insert one more observation node, reveal either signal K or J, and then move to two different terminal nodes with fixed utilities -1 and $+1$. These TFSDPs are

structurally different but strategically equivalent, and the core idea of CFR is unchanged under either representation.

1.2 Regret Matching

Regret Matching (RM) is one of the simplest no-regret procedures. Suppose a player repeatedly chooses a mixed action $x^t \in \Delta^{|A|}$ and then observes a utility vector $\ell^t \in \mathbb{R}^{|A|}$. The cumulative regret for action a after T rounds is

$$R^T(a) = \sum_{t=1}^{T} \left(\ell^t(a) - \langle x^t, \ell^t \rangle \right).$$

RM chooses the next strategy by normalizing only the positive cumulative regrets:

$$x^{T+1}(a) = \begin{cases} \dfrac{[R^T(a)]_+}{\sum_{b \in A} [R^T(b)]_+}, & \text{if } \sum_{b \in A} [R^T(b)]_+ > 0, \\ \dfrac{1}{|A|}, & \text{otherwise,} \end{cases}$$

where $[r]_+ = \max\{r, 0\}$.

The algorithm is remarkably simple: maintain cumulative regrets, keep only the positive part, normalize, and repeat. Regret Matching+ (RM+) modifies the update by truncating cumulative regrets at zero after every iteration. In repeated two-player zero-sum play, the average strategies induced by RM converge to a Nash equilibrium. A classical proof of this fact is obtained through **Blackwell's approachability theorem**.

1.3 CFR and the Regret-Circuit Intuition

Counterfactual Regret Minimization (CFR) extends the same idea to imperfect-information games. The original algorithm was introduced by Zinkevich et al. [2]. Rather than minimizing regret over the entire normal-form strategy space directly, CFR decomposes the problem across information sets. Each information set is assigned a local regret minimizer, typically RM or RM+, and these local modules are wired together through the game tree.

This is exactly the viewpoint emphasized by regret circuits [3]. Each local module receives a linear utility vector computed from the rest of the tree, updates its own regrets, and outputs a behavioral strategy for that information set. The global strategy is then the composition of all these local decisions.

For player i, strategy profile σ, and information set I, the **counterfactual utility** can be written as

$$u_i(\sigma, I) = \frac{\sum_{h \in I} \sum_{z \in Z : h \sqsubset z} \pi^{\sigma}_{-i}(h)\, \pi^{\sigma}(h, z)\, u_i(z)}{\pi^{\sigma}_{-i}(I)}.$$

This is the quantity referred to in [2] as "the expected utility given that information set I is reached and all players play using strategy σ except that player i plays to reach I." The precise reason this formula has that meaning is discussed in the next section.

The instantaneous counterfactual regret for action $a \in A(I)$ is the difference between the counterfactual utility of deviating to a at I and the counterfactual utility of following the current strategy at I. CFR accumulates these local regrets over time and applies regret matching independently at each information set. In two-player zero-sum games, the average strategy converges to equilibrium.

2 Clarifications and Implementation Details

This section collects points that were not fully clear to me from the paper or lecture notes alone, and that became clearer only after working through the notation and the implementation details.

2.1 A Notation Subtlety About Reach Probabilities

One point that caused confusion while reading [2] was the notation for **reach probabilities**. For a history h, let $\pi^\sigma(h)$ denote the probability that h is reached under strategy profile σ. This can be decomposed into player i's contribution and the contribution of everyone else (including chance):

$$\pi^\sigma(h) = \pi_i^\sigma(h)\pi_{-i}^\sigma(h).$$

Here $\pi_i^\sigma(h)$ is the probability that, at every prefix of h where player i acts, player i chooses the action specified by h, and $\pi_{-i}^\sigma(h)$ is the product of the corresponding contributions from the opponents and chance. At the history level, this factorization is completely natural.

For an information set I, [2] defines

$$\pi^\sigma(I) = \sum_{h \in I} \pi^\sigma(h),$$

and then states that $\pi_i^\sigma(I)$ and $\pi_{-i}^\sigma(I)$ are "defined similarly." This leaves the intended definitions ambiguous. A direct symmetric extension would be

$$\pi_i^\sigma(I) = \sum_{h \in I} \pi_i^\sigma(h), \qquad \pi_{-i}^\sigma(I) = \sum_{h \in I} \pi_{-i}^\sigma(h).$$

However, these are not the correct definitions if one wants the information-set version of the history-level factorization

$$\pi^\sigma(I) = \pi_i^\sigma(I)\pi_{-i}^\sigma(I).$$

Indeed, if I is an information set of player i, then perfect recall implies that $\pi_i^\sigma(h)$ is the same for every $h \in I$. Let this common value be c. Then

$$\pi^\sigma(I) = \sum_{h \in I} \pi_i^\sigma(h)\pi_{-i}^\sigma(h) = c \sum_{h \in I} \pi_{-i}^\sigma(h),$$

whereas the symmetric extension gives

$$\pi_i^\sigma(I)\pi_{-i}^\sigma(I) = \left(\sum_{h \in I} \pi_i^\sigma(h)\right)\left(\sum_{h \in I} \pi_{-i}^\sigma(h)\right) = |I|\, c \sum_{h \in I} \pi_{-i}^\sigma(h),$$

which is not equal to $\pi^\sigma(I)$ unless $|I| = 1$.

The interpretation consistent with perfect recall and the counterfactual-utility formula is therefore

$$\pi_i^\sigma(I) = \pi_i^\sigma(h) \qquad \text{for any } h \in I,$$

and

$$\pi_{-i}^\sigma(I) = \sum_{h \in I} \pi_{-i}^\sigma(h).$$

With this definition,

$$\pi^\sigma(I) = \sum_{h \in I} \pi_i^\sigma(h)\pi_{-i}^\sigma(h) = \pi_i^\sigma(I) \sum_{h \in I} \pi_{-i}^\sigma(h) = \pi_i^\sigma(I)\pi_{-i}^\sigma(I).$$

This is the definition that matches the history-level factorization and makes the later counterfactual-utility formula work.

This interpretation also explains why the verbal definition of counterfactual utility matches the formula used in [2]. If we start from the conditional-expectation view

$$u_i(\sigma, I) = \frac{\sum_{h \in I} \sum_{z \in Z : h \sqsubset z} \pi^\sigma(h)\, \pi^\sigma(h, z)\, u_i(z)}{\sum_{h \in I} \pi^\sigma(h)},$$

and then factor $\pi^\sigma(h) = \pi_i^\sigma(h)\pi_{-i}^\sigma(h)$, we obtain

$$u_i(\sigma, I) = \frac{\sum_{h \in I} \sum_{z \in Z : h \sqsubset z} \pi_i^\sigma(h)\pi_{-i}^\sigma(h)\pi^\sigma(h, z)u_i(z)}{\sum_{h \in I} \pi_i^\sigma(h)\pi_{-i}^\sigma(h)}.$$

Since $\pi_i^\sigma(h)$ is the same for all $h \in I$, it can be pulled out of both the numerator and denominator and canceled. Therefore

$$u_i(\sigma, I) = \frac{\sum_{h \in I} \sum_{z \in Z : h \sqsubset z} \pi_{-i}^\sigma(h)\pi^\sigma(h, z)u_i(z)}{\sum_{h \in I} \pi_{-i}^\sigma(h)} = \frac{\sum_{h \in I} \sum_{z \in Z : h \sqsubset z} \pi_{-i}^\sigma(h)\pi^\sigma(h, z)u_i(z)}{\pi_{-i}^\sigma(I)},$$

which is exactly the standard counterfactual-utility expression. This cancellation is the precise reason the notation for $\pi_i^\sigma(I)$ must be handled carefully.

2.2 Implementation Details

Several practical difficulties arose when translating the CFR pseudocode into an implementation. Even with the fairly explicit description of CFR in Farina's Lecture 5 [4], the implementation was not immediate. First, the linear utility vector ℓ^t passed to the local regret minimizers is not given directly by an EFG representation; it has to be constructed by traversing the tree and correctly incorporating the opponent's reach probabilities. Second, the role of the auxiliary quantity V^t in the bottom-up dynamic program is not obvious from the pseudocode alone. In the implementation, the most useful interpretation was that $V^t[v]$ represents the value of node v with the relevant reach-probability factor already folded in. Since ℓ^t is induced directly only from terminal utilities, the purpose of the bottom-up dynamic program is to propagate these weighted values upward and fill in the corresponding values for all internal nodes.

3 Experiments

All algorithms and experiments in this report were implemented in Python. The source code and notebooks are available at `https://github.com/swhur1214/cfr`.

3.1 Notebook 1: RM on Rock-Paper-Scissors

The first experiment applies RM and RM+ to Rock-Paper-Scissors (RPS), a two-player zero-sum matrix game with a unique mixed Nash equilibrium at the uniform strategy. Since a uniform initial strategy is already an equilibrium, starting from zero regret would make the algorithm remain there immediately. To obtain a more informative learning curve, I initialized the cumulative regrets with non-uniform values and then ran self-play for 10,000 iterations.

Exploitability was measured as NashConv/2. The final results were:

Algorithm	Player 0 average strategy	Player 1 average strategy	Final exploitability
RM	$(0.3392, 0.3343, 0.3265)$	$(0.3302, 0.3315, 0.3383)$	0.00977403
RM+	$(0.3320, 0.3328, 0.3352)$	$(0.3348, 0.3338, 0.3314)$	0.00287796

Empirically, both methods showed a convergence trend roughly consistent with $O(1/\sqrt{T})$, and RM+ converged faster, as expected. However, an interesting feature of both curves is that they oscillate rather than decrease monotonically. My interpretation is that this is related to the rotational dynamics of RPS: instantaneous play naturally cycles around the mixed equilibrium, and averaging only partially smooths this effect. This suggests that purely first-order regret updates may be slowed down by cyclic behavior, and it would be interesting to compare them with more predictive or higher-order updates in the future.

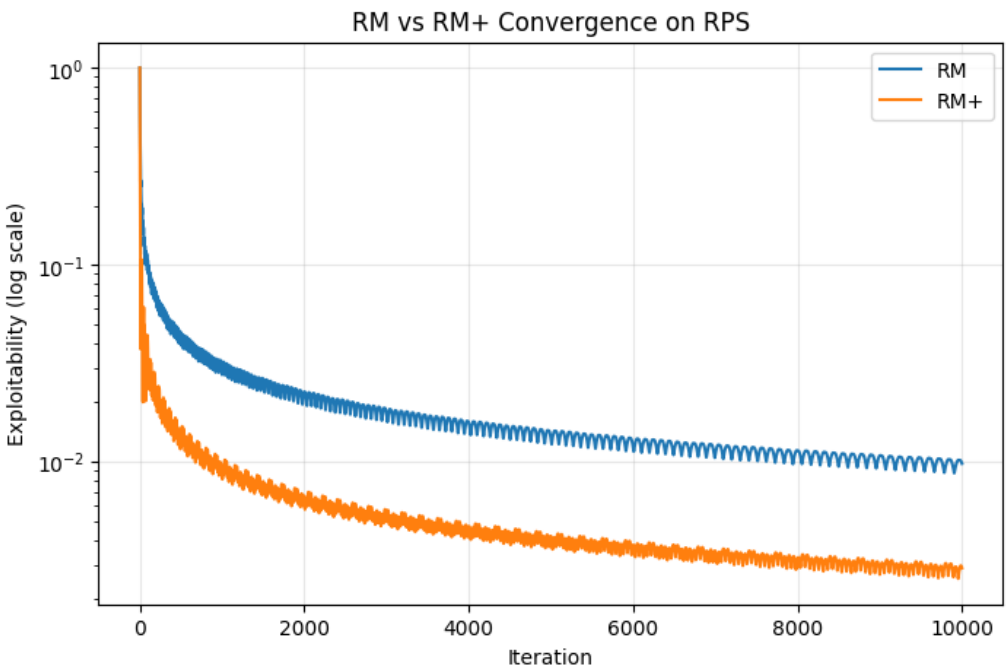

Figure 1: Exploitability of the average strategy for RM and RM+ on Rock-Paper-Scissors.

3.2 Notebook 2: CFR on Kuhn Poker

The second experiment applies CFR and CFR+ to Kuhn Poker. In the implementation, the initial chance event is represented as a single node with six equally likely card deals. This choice is strategically equivalent to a deeper representation that deals cards one by one, but it is more compact and keeps the game tree easy to inspect.

Again I ran 10,000 iterations and measured exploitability of the average strategy. The final results were:

Algorithm	Final value for Player 0	Final exploitability
CFR	-0.055548	0.00232917
CFR+	-0.055546	0.00130270

Both algorithms converged to strategies with very small exploitability, and CFR+ again converged faster than the vanilla version. The curves also showed oscillatory behavior rather than a perfectly

monotone decrease, similar to the RPS experiment. The learned game value is also extremely close to the known value of Kuhn Poker, $-1/18 \approx -0.055556$, which provides a useful sanity check on the implementation. Qualitatively, the learned strategies also match the standard structure of the equilibrium: bluffing with the weakest card at a moderate frequency, betting aggressively with the strongest card, and mixing in the middle with the queen in response situations.

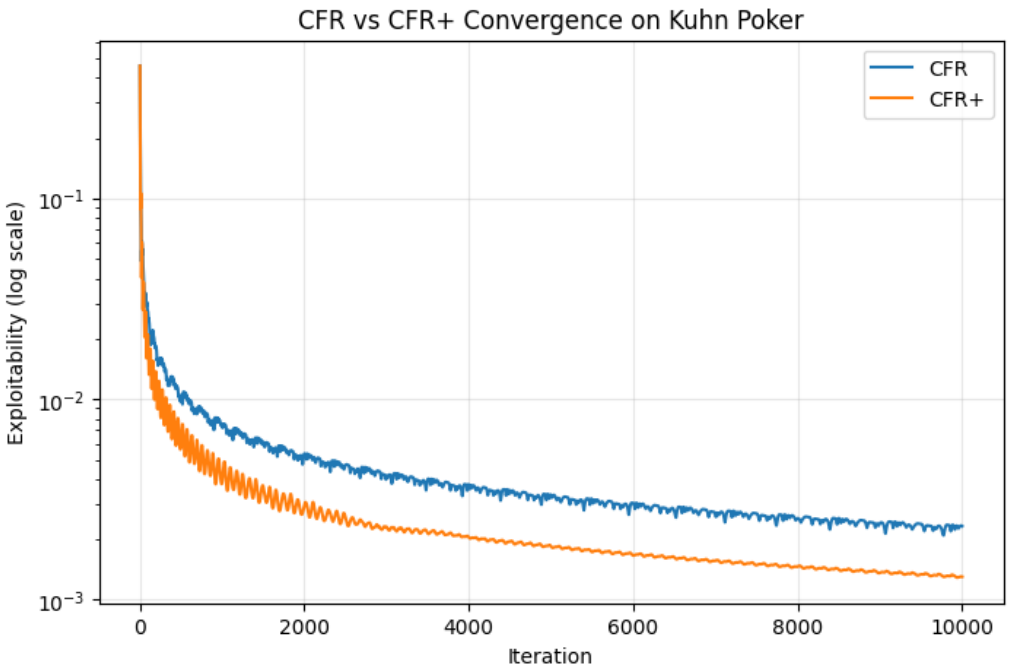

Figure 2: Exploitability of the average strategy for CFR and CFR+ on Kuhn Poker.

4 Conclusion

This project helped clarify two complementary ideas. First, despite its simplicity, RM finds optimal mixed strategies. Second, CFR can be understood as wiring together many such local regret minimizers over an imperfect-information game tree. That viewpoint makes the algorithm substantially more transparent than it first appears in the original paper. Implementing the algorithm, rather than only reading the paper and lecture notes, was especially instructive because it forced me to resolve several gaps between the abstract pseudocode and an actual EFG-based implementation.

From the experimental side, both RM-type and CFR-type methods showed oscillatory learning curves, suggesting that a method for reducing this oscillation could improve convergence speed in practice.

A natural next step is to study Monte Carlo CFR (MCCFR) and abstraction methods such as bucketing. These techniques are widely regarded as essential for scaling from toy games such as Kuhn Poker to realistic poker variants.

References

[1] *Kuhn poker*. Wikipedia. `https://en.wikipedia.org/wiki/Kuhn_poker`. Accessed March 7, 2026.

[2] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems 20*, 2007.

[3] Gabriele Farina, Christian Kroer, and Tuomas Sandholm. Regret circuits: Composability of regret minimizers. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[4] Gabriele Farina. *Computational Game Solving (CMU 15-888), Lectures 1–5*. 2021.